

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

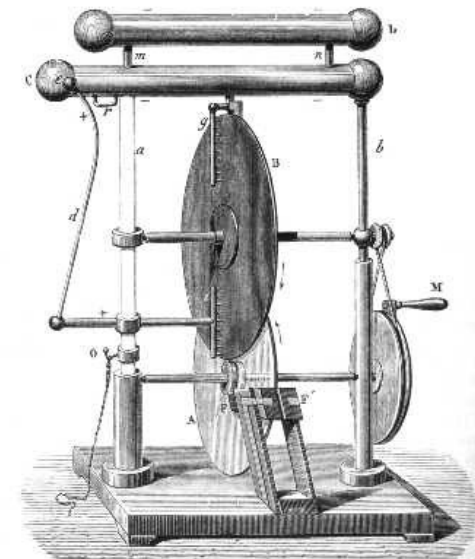
Prof. Dr. Stephan Diehl
Informatik, Universität Trier

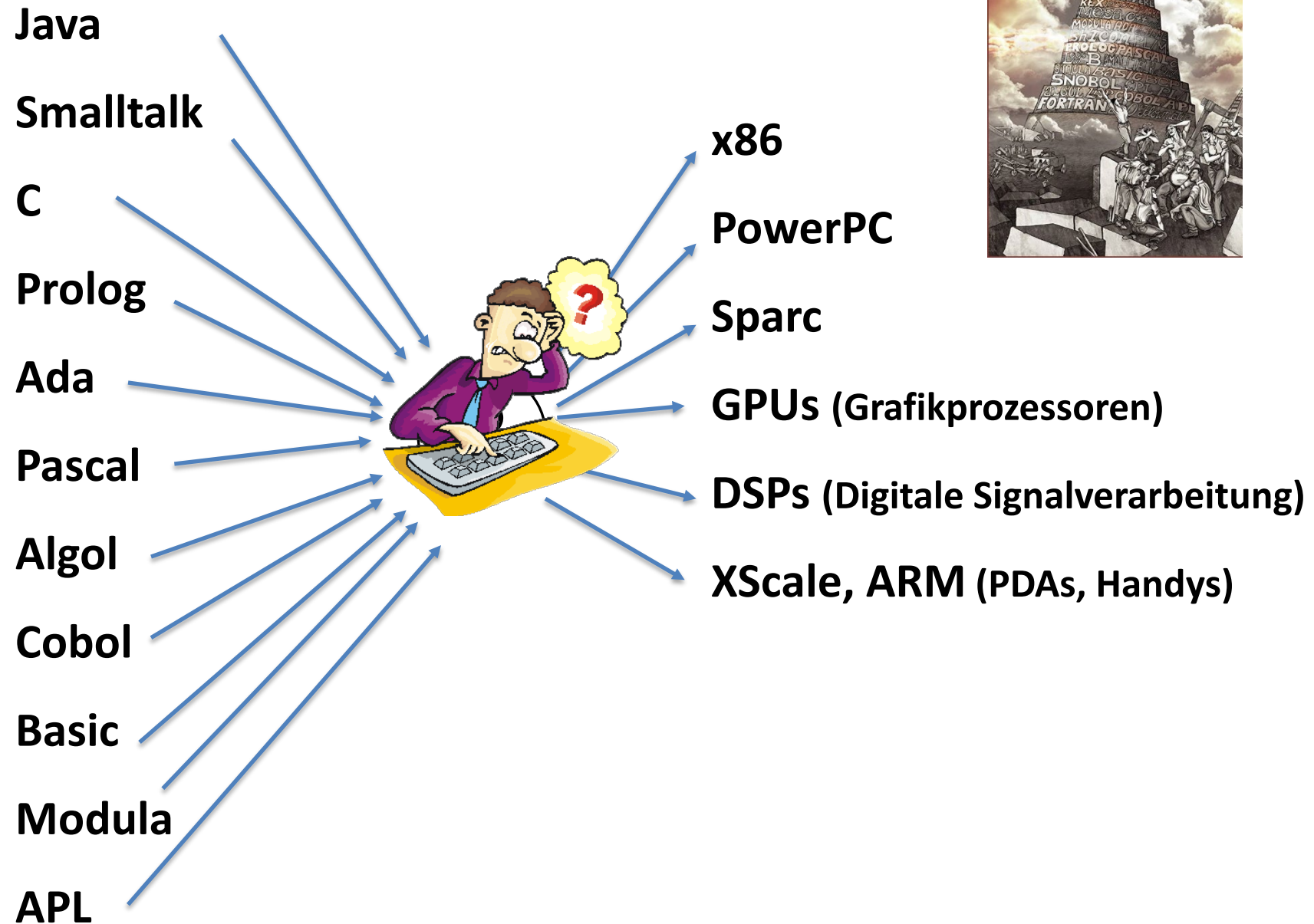
Organisatorisches

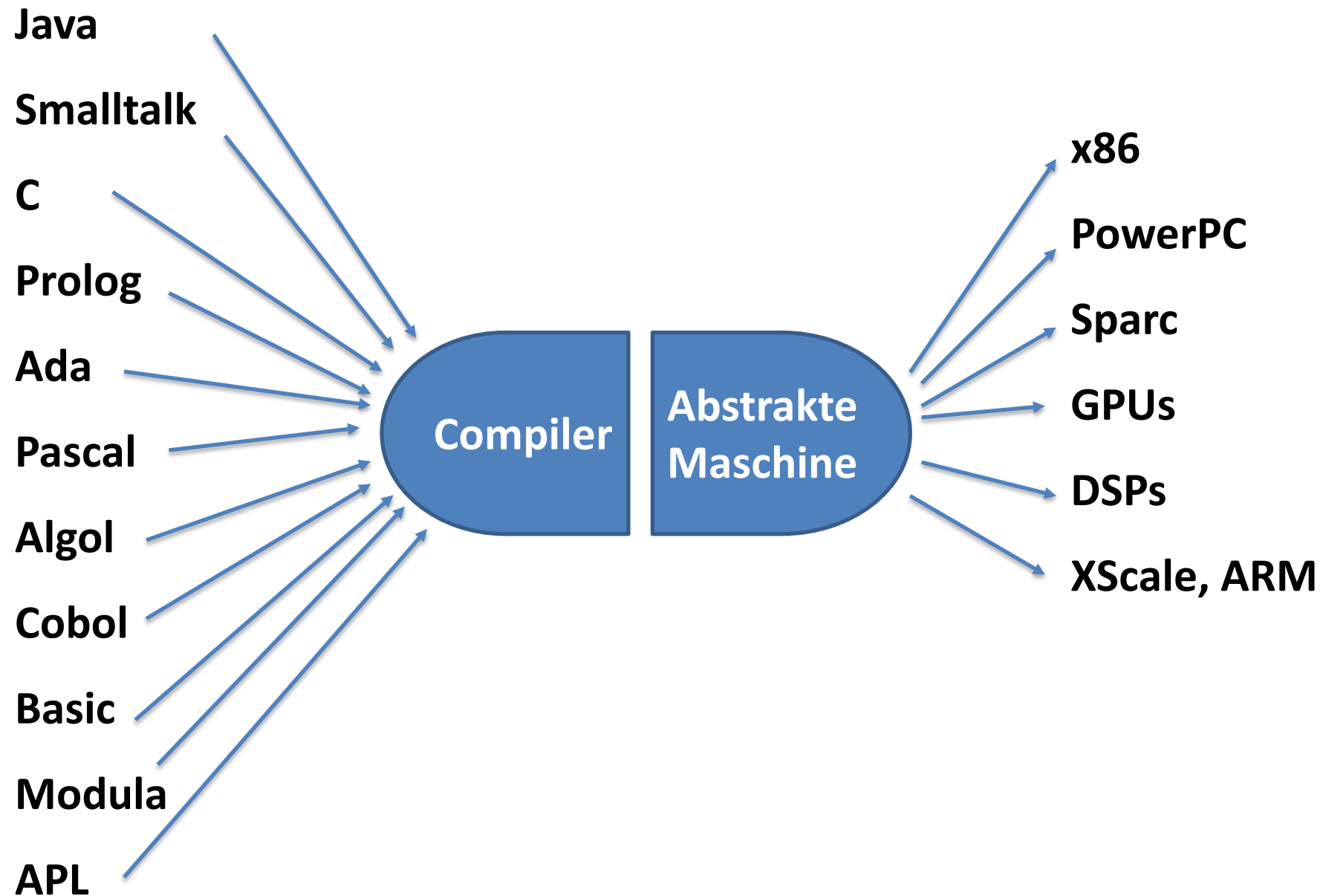
- **Vorlesung:**
 - Di, 14-16, H 507
 - Do, 12-14, H 507
- **Übungen:**
 - **Dienstag, 16-18 Uhr** s.t. in H 507
 - Die erste Übung wird am Donnerstag, dem **28.10.2025** stattfinden.

Heute

- Abstrakte Maschinen
 - Um die Projektarbeit zeitlich über das Semester zu verteilen, fangen wir mit abstrakten Maschinen an.
 - Wir werden später in der Vorlesung wieder auf dieses Thema zurückkommen, dann schauen wir uns die Java Virtual Machine genauer an.







PHASE 1

Intermediate
representation**Code
generation**abstract machine
code

PHASE 2

abstract machine
code**Compiler**concrete machine
code

alternatively:



Input

**Interpreter**

Output

Abstract Machines

- „ Abstract machines are machines because they **permit step-by-step execution** of programs; they are abstract because they **omit the many details** of real (hardware) machines. Abstract machines provide an **intermediate language stage for compilation**. They bridge the gap between the high level of a programming language and the low level of a real machine. The instructions of an abstract machine are **tailored** to the particular operations required to implement operations of a specific source language or class of source languages.”

Quelle: Abstract machines for programming language implementation,
Stephan Diehl, Pieter Hartel, Peter Sestoft; Future Generation Computer Systems 16, 2000.

Abstrakte Maschinen

- Idealisierte Architektur
 - erleichtert insbesondere die Übersetzung von Programmiersprachen, deren Berechnungsmodell stark von dem realer Prozessoren abweicht
- Einfache Kodeerzeugung (Phase 1)
- Leicht auf realen Prozessoren zu implementieren
- Erleichtern das Portieren eines Compilers auf neue Zielarchitekturen (Phase 2)
- Diese Modularisierung erleichtert Änderungen des Compilers (ähnlich der Anpassung des Backends)
- **Übersetzung der Sprachkonstrukte ist getrennt von den Optimierungen, die die Architektureigenschaften der Zielplattform ausnutzen**

Einige Programmiersprachen und ihre abstrakten Maschinen

Sprache	Abstrakte Maschine
Pascal	P-Machine
Smalltalk	Smalltalk Virtual Machine
Prolog	Warren Abstract Machine (WAM)
SML, Haskell	Spineless Tagless Graph Reduction Machine (STGM)
Java, Scala	Java Virtual Machine (JVM)

Beispiele

- Anzeige von AM-Code in Java

```
javap -c Beispiel.class
```

- Anzeige von AM-Code in Python:

```
import dis
```

```
def f(n) :  
    return n-1;
```

```
dis.dis(f)
```

Bemerkungen

- Typische Eigenschaften einer abstrakten Maschine:
 - Register, Speicher, Keller, Instruktionsvorrat, Programmspeicher
- Operationelle Semantik in Form einer mathematischen Maschine
 - Für eine Programmiersprache wie Java zu kompliziert, allerdings gibt es Versuche, auch die Semantik von Java formal zu spezifizieren.

Die Mutter aller Kellermaschinen: MaMa

Die Kellermaschine Maschine MaMa ist ein Tupel

$\text{MaMa} = (K, P, \text{step})$ mit

- Konfigurationsmenge $K \in \mathbb{N} \times \mathbb{N} \times (\mathbb{N} \rightarrow \mathbb{N})$,
- Programmspeicher $P \in \mathbb{N} \rightarrow \mathbb{I}$
- und der Übergangsfunktion $\text{step} \in K \rightarrow K$.

Eine Konfiguration $(s, p, S) \in K$ besteht aus dem Kellerzeiger (Stackpointer) s , dem Programmzähler p und dem Keller (Stack) S .

Rechnung der MaMA

- $K_0 \triangleright K_n \Leftrightarrow \exists K_0, \dots, K_n$ mit $\text{step}(K_i) = K_{i+1}$
- Die Folge $K_0, \dots, K_n$ heißt **Rechnung** von MaMa mit Startkonfiguration K_0
- Die Berechnung von MaMa **terminiert** $\Leftrightarrow \nexists K_m$ mit $K_n \triangleright K_m$
- In diesem Fall benötigt die Rechnung Zeit n
- Die Berechnung ist **deterministisch**, da step eine Funktion ist.

Übergangsfunktion

- Die Menge I der Instruktionen ist bisher nicht genauer definiert.
- Beispiel:
 - $I = I_0 \cup I_1 \times \mathbb{N}$
 - wobei $I_0 = \{\text{stop, pop, add, sub, ...}\}$ nullstellige und $I_1 = \{\text{push}(n), \text{equal}(n), \text{ujp}(n), \text{ldo}(n), \dots \mid n \in \mathbb{N}\}$ einstellige Instruktionen sind.
 - $\text{step}(s, p, S) = (s+1, p+1, S[s+1 \setminus n])$ falls $P(p) = \text{push}(n)$
 - $\text{step}(s, p, S) = (s-1, p+1, S[s-1 \setminus S(s-1) + S(s)])$ falls $P(p) = \text{add}$
 - ...

Notation:

Überschreiben einer Funktion f an einer Stelle

$$f[x \backslash v](y) = \begin{cases} v & \text{falls } y=x \\ f(y) & \text{sonst} \end{cases}$$

Alternative Definition:

$$f[x \backslash v] = f' \text{ mit}$$

$$f'(y) = \begin{cases} v & \text{falls } y=x \\ f(y) & \text{sonst} \end{cases}$$

Eine Übergangsfunktion

step(s,p,S)=

- $(s+1, p+1, S[s+1 \setminus n])$ falls $P(p) = \text{push}(n)$
- $(s-1, p+1, S)$ falls $P(p) = \text{pop}$
- $(s-1, p+1, S[s-1 \setminus S(s-1) + S(s)])$ falls $P(p) = \text{add}$
- $(s-1, p+1, S[s-1 \setminus S(s-1) / S(s)])$ falls $P(p) = \text{div}$
- $(s+1, p+1, S[s+1 \setminus s])$ falls $P(p) = \text{ldsp}$
- $(s-2, a, S)$ falls $P(p) = \text{equal}(a)$ und $S(s-1) = S(s)$
- $(s-2, p+1, S)$ falls $P(p) = \text{equal}(a)$ und $S(s-1) \neq S(s)$
- $(s+1, p+1, S[s+1 \setminus S(s+n)])$ falls $P(p) = \text{ldo}(n)$
- $(s-2, a, S)$ falls $P(p) = \text{leq}(a)$ und $S(s-1) \leq S(s)$
- $(s-2, p+1, S)$ falls $P(p) = \text{leq}(a)$ und $S(s-1) > S(s)$
- $(s-1, p+1, S[s+n \setminus S(s)])$ falls $P(p) = \text{sto}(n)$
- (s, a, S) falls $P(p) = \text{ujp}(a)$
- undefiniert falls $P(p) = \text{stop}$

ähnlich: sub,
mult und mod

Problem und Spezifikation

m ist kleinstes Element einer Liste L von Zahlen $\Leftrightarrow \nexists n \in L: n < m$

- m ist nicht eindeutig bestimmt!
- Wir nehmen an L ist endlich!
- Für Algorithmus hilfreich
- m kleinstes Element von $L \Leftrightarrow \forall n \in L: m \leq n$

Algorithmus und Programm

Startkonfiguration:

$S(0)=n_1, \dots, S(k-1)=n_k$ und $s=k-1, p=0$

- 1) Sei $L=[n_1, \dots, n_k]$
- 2) $m=n_1$
- 3) $i=2$;
- 4) Falls $n_i < m$ dann $m=n_i$
- 5) Erhöhe i um 1
- 6) Falls $i < k$ gehe zu 4)
- 7) Fertig! m ist kleinstes Element der Liste.

- $P(0) = \text{ldsp}$
 - $P(1) = \text{push}(0)$
 - $P(2) = \text{equal}(10)$
 - $P(3) = \text{ldo}(-1)$
 - $P(4) = \text{ldo}(-1)$
 - $P(5) = \text{leq}(8)$
 - $P(6) = \text{sto}(-1)$
 - $P(7) = \text{ujp}(0)$
 - $P(8) = \text{pop}$
 - $P(9) = \text{ujp}(0)$
 - $P(10) = \text{stop}$
-

Startkonfiguration: $S(0)=n_1, \dots, S(k-1)=n_k$
und $s=k-1, p=0$

- $P(0) = \text{ldsp}$
 - $P(1) = \text{push}(0)$
 - $P(2) = \text{equal}(10)$
 - $P(3) = \text{ldo}(-1)$
 - $P(4) = \text{ldo}(-1)$
 - $P(5) = \text{leq}(8)$
 - $P(6) = \text{sto}(-1)$
 - $P(7) = \text{ujp}(0)$
 - $P(8) = \text{pop}$
 - $P(9) = \text{ujp}(0)$
 - $P(10) = \text{stop}$
-

$(2,0,(5,4,6))$		
$(3,1,(5,4,6,2))$	ldsp	1. Iteration: 8 Schritte
$(4,2,(5,4,6,2,0))$	push(0)	
$(2,3,(5,4,6))$	equal(10)	
$(3,4,(5,4,6,4))$	ldo(-1)	
$(4,5,(5,4,6,4,6))$	ldo(-1)	
$(2,8,(5,4,6))$	leq(8)	
$(1,9,(5,4))$	pop	
$(1,0,(5,4))$	ujp(0)	
$(2,1,(5,4,1))$	ldsp	2. Iteration: 8 Schritte
$(3,2,(5,4,1,0))$	push(0)	
$(1,3,(5,4))$	equal(10)	
$(2,4,(5,4,5))$	ldo(-1)	
$(3,5,(5,4,5,4))$	ldo(-1)	
$(1,6,(5,4))$	leq(8)	
$(0,7,(4))$	sto(-1)	
$(0,0,(4))$	ujp(0)	
$(1,1,(4,0))$	ldsp	Letzte Iteration: 4 Schritte
$(2,2,(4,0,0))$	push(0)	
$(0,10,(4))$	equal(10)	
$(0,10,(4))$	halt	
Keine Folgekonfiguration		Insgesamt: 20 Schritte

Allgemein Laufzeit: $(k-1)*8 + 4$ Schritte

Laufzeit des Programms

- Für Liste [3, 2, 5]
- Allgemein? Für Liste der Länge k ?
 - Best case, average case, worst case ?
 - Glücklicherweise hier alle gleich!
 - Laufzeit: $4+(k-1)*8$